

0.81.2. Ramasser une pièce : le P — au hasard

Le 0.81.1, mais la nouvelle place est tirée au hasard : `hasard() % 20` pour la colonne, `hasard() % 17` pour la ligne.

1. Ce qu'il faut comprendre

C'est le 0.81.1, sans le tableau : la nouvelle place est tirée **au hasard**.

Ce qui est nouveau ici : `hasard()`. Elle rend un nombre imprévisible, de 0 à 255. `hasard() % 20` en garde le **reste** de la division par 20 : un nombre de 0 à 19, une colonne. `hasard() % 17` : une ligne de 0 à 16 (la ligne 17 est celle du score).

On ne sait jamais où la pièce va tomber : c'est ce qui rend un jeu différent à chaque partie.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1919 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- LES VARIABLES : tout ce que le jeu doit retenir ----
```

2. UNE VARIABLE — X

```
uint8_t x = 5;           // la COLONNE du A (0 à gauche ... 19 à droite) : il part de la colonne 5
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;           // la LIGNE du A (0 en haut ... 17 en bas) : il part de la ligne 8
```

4. UNE VARIABLE — PX

```
uint8_t px = 15;         // la colonne de la PIÈCE. La pièce est une LETTRE : le P.
```

5. UNE VARIABLE — PY

```
uint8_t py = 8;           // sa ligne. Elle est donc en (15, 8), sur la même ligne que le A.
```

6. UNE VARIABLE — SCORE

```
uint8_t score = 0;        // combien de pièces on a ramassées : 0 au départ
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(px, py, ALPHABET[15]);
    texte(0, 17, "SCORE");

    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250);

        if (x == px && y == py) {           // le A est sur la pièce (le 0.81)
            score = score + 1;

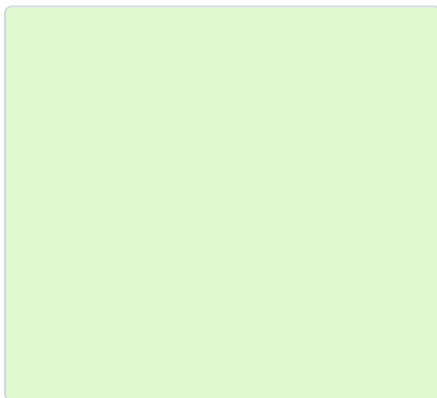
            // La nouvelle place, AU HASARD :
            //
            //   px = hasard() % 20;
            //   |           |
            //   |           +-- % 20 : le reste de la division par 20, de 0 à 19.
            //   |           Exemple : hasard() rend 137 ; 137 = 6 × 20 + 17 ;
            //   |           le reste est 17 → la colonne 17.
            //   +----- un nombre imprévisible, de 0 à 255
            px = hasard() % 20;               // une colonne : 0 à 19
            py = hasard() % 17;               // une ligne : 0 à 16 (la 17, c'est le score)

            poser(px, py, ALPHABET[15]);     // la nouvelle pièce apparaît là
        }

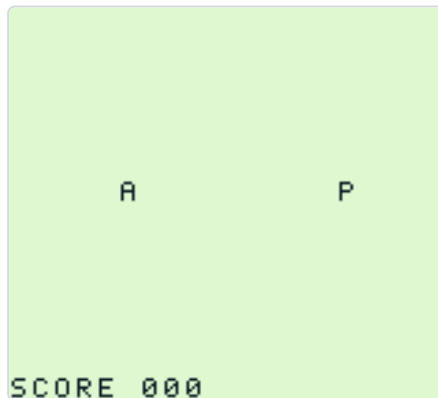
        nombre(6, 17, score);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le A ramasse le P ; un autre P apparaît quelque part au hasard.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la première pièce est ramassée
- ✓ une nouvelle pièce est quelque part
- ✓ dans l'écran

Ensuite : **0.82. Un mur qu'on ne traverse pas : le M** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++